

PHONEX-C⁰

A Minimal Persistent Transformer Implemented in C99

<https://github.com/aam-007/phonex-c0>

16th January, 2026

ADITYA MISHRA

ABSTRACT

Modern transformer models are typically developed using high-level frameworks that abstract away core computational and systems-level details. While effective for rapid scaling, these abstractions obscure the fundamental mechanisms underlying attention, normalization, backpropagation, and parameter persistence. This paper introduces Phonex-C⁰, a minimal, fully self-contained implementation of a causal transformer written entirely in standard C99. The system supports end-to-end training and inference, explicit forward and backward passes, and binary serialization of model parameters for persistent reuse. By eliminating external dependencies and exposing all numerical operations explicitly, Phonex-C⁰ serves both as an educational reference and as a foundation for low-level experimentation in language modeling, systems research, and embedded learning environments.

1. INTRODUCTION

Transformer architectures have become the dominant paradigm for sequence modeling across natural language processing and related domains. Despite their widespread adoption, most practical implementations rely on deeply layered software stacks that conceal critical implementation details such as tensor layout, numerical stability, gradient flow, and memory management. As a result, understanding transformers at a mechanistic level remains challenging, particularly for researchers and engineers interested in performance, portability, or correctness at the lowest levels of abstraction.

Phonex-C⁰ is designed to make the transformer transparent. It implements a complete causal transformer block from first principles using only standard C libraries. All components, including embeddings, masked self-attention, layer normalization, feed-forward networks, softmax, loss computation, and backpropagation, are written explicitly. No automatic differentiation, external tensor engines, or numerical libraries are used. Every intermediate activation and gradient is materialized and managed manually.

A distinguishing feature of Phonex-C⁰ is its built-in persistence layer. Model parameters can be serialized into a compact binary format and reloaded for inference or continued training. This enables stateless execution, reproducibility, and inspection of learned weights without relying on framework-specific checkpoint formats. Training and inference are controlled through a minimal command-line interface, further reinforcing the system's focus on clarity and portability.

The primary objective of this work is not performance or scale, but precision of understanding. By constraining the model size

and implementation scope, Phonex-C⁰ exposes the exact mechanics of transformer learning while remaining executable on commodity hardware. The resulting system provides a reference implementation for low-level transformer design and a platform for experimenting with numerical methods, optimization strategies, and persistent model storage in resource-constrained environments.

2. CONTRIBUTIONS

The contributions of this paper are as follows:

1. A complete causal transformer implementation written entirely in standard C99.
2. Explicit forward and backward passes for all model components without automatic differentiation.
3. A binary serialization format enabling persistent storage and reloading of model parameters.
4. A minimal command-line interface supporting training and inference in a single executable.
5. A reference system suitable for education, systems research, and low-level experimentation.

3. MODEL ARCHITECTURE

Phonex-C0 implements a single-block causal transformer designed to model character-level sequences. The architecture follows the canonical transformer structure while remaining intentionally minimal. The model operates on fixed-length input sequences and uses a fixed vocabulary size corresponding to the ASCII character set.

Each input token is mapped to a learned embedding vector of dimension D_{model} .

Positional information is incorporated through a learned positional embedding, which is added directly to the token embedding. This sum forms the initial representation passed to the transformer block.

The transformer block consists of two layer normalization stages, a masked self-attention mechanism, and a position-wise feed-forward network. Residual connections are applied after both the attention and feed-forward sublayers. The output of the block is projected to vocabulary logits through a final linear transformation, followed by a softmax to obtain token probabilities.

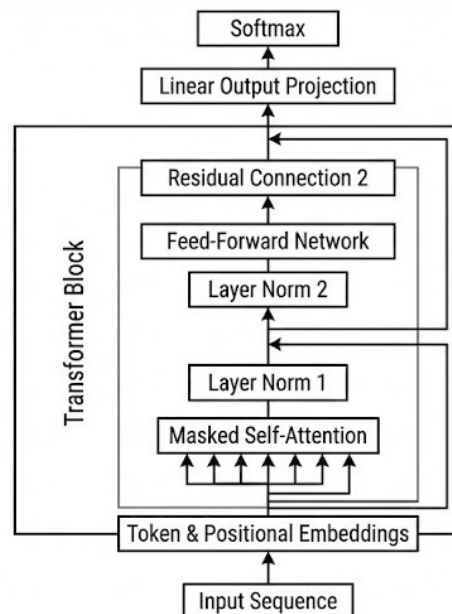


Figure 1: PHONEX-C0 High-Level Architecture (Single Block Causal Transformer)

4. TENSOR REPRESENTATION & MEMORY LAYOUT

All computations in Phonex-C0 are expressed in terms of a simple tensor abstraction. A tensor is defined as a contiguous block of memory storing floating-point values along with a corresponding gradient buffer of equal size.

Each tensor explicitly tracks its row and column dimensions.

Memory allocation and deallocation are performed manually using standard heap allocation. No dynamic resizing occurs during execution. Instead, all tensors required for forward and backward propagation are allocated once at initialization. This design ensures deterministic memory usage and avoids runtime allocation overhead during training.

Gradients are accumulated explicitly in each tensor's gradient buffer. Before every training step, all gradients are reset to zero. This approach mirrors the behavior of modern deep learning frameworks while remaining fully transparent to the user.

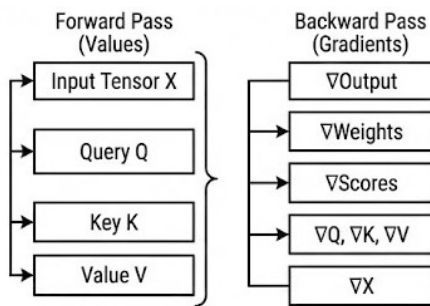


Figure 2: Explicit Tensor Memory Layout & Gradient Flow (Attention Mechanism Example)

5. FORWARD COMPUTATION

5.1 Embedding and Input Construction

For a given input sequence of token indices, the model retrieves the corresponding token embeddings and positional embeddings. These vectors are summed element-wise to produce the input representation for each time step.

This operation is implemented as a direct memory access and addition, avoiding any intermediate abstractions. Invalid or out-of-

range token indices are clamped to ensure numerical safety.

5.2 Layer Normalization

Layer normalization is applied across the feature dimension at each time step. For each token representation, the mean and variance are computed explicitly. The normalized activations are then scaled and shifted using learned gain and bias parameters.

To ensure numerical stability, a small constant is added to the variance before inversion. Intermediate statistics such as the per-token mean and variance are stored and reused during the backward pass.

5.3 Masked Self-Attention

Self-attention is implemented using learned projection matrices for queries, keys, and values. For each time step, the input representation is projected into query, key, and value spaces through explicit matrix multiplication.

Causal masking is enforced by setting attention scores for future positions to a large negative constant prior to softmax normalization. This guarantees that each token can attend only to itself and preceding tokens.

Attention scores are scaled by the inverse square root of the model dimension to control gradient magnitude. The softmax operation is implemented with numerical stabilization by subtracting the maximum score before exponentiation.

The attention probabilities are used to compute a weighted sum of value vectors, which is then projected through an output matrix to produce the attention output.

5.4 Feed-Forward Network

The feed-forward network consists of two linear transformations with a ReLU nonlinearity in

between. The first transformation expands the feature dimension, and the second projects it back to the model dimension.

The ReLU activation is applied element-wise, and its binary mask is implicitly encoded through the sign of the pre-activation values. This information is used during backpropagation to gate gradients.

5.5 Output Projection and Softmax

The final output of the transformer block is projected into the vocabulary space using a linear transformation. A softmax function converts logits into probability distributions over the vocabulary.

The model is trained using a cross-entropy loss computed between the predicted probabilities and the target next-token indices.

6. BACKWARD Computation

Phonex-C0 implements full backpropagation manually for every component of the model. Gradients are propagated in the exact reverse order of the forward pass.

6.1 Softmax and Loss Gradient

The gradient of the cross-entropy loss with respect to the logits is computed analytically as the difference between the predicted probability and the one-hot encoded target. This avoids the need to backpropagate explicitly through the softmax function.

6.2 Linear Layers

For all linear transformations, gradients with respect to weights, biases, and inputs are computed using explicit nested loops. Weight gradients are accumulated as outer products of inputs and output gradients. Input gradients are computed as weighted sums of output gradients and weight values.

6.3 Attention Backpropagation

Backpropagation through the attention mechanism is implemented in stages. Gradients are propagated from the attention output through the output projection, into the value vectors, attention probabilities, and attention scores.

The softmax gradient is computed explicitly using the Jacobian structure of the softmax function. Gradients are then propagated into the query and key projections, taking into account the scaling factor applied during the forward pass.

Finally, gradients are accumulated back into the original input representations through the query, key, and value projection matrices.

6.4 Layer Normalization Back propagation

Gradients through layer normalization are computed using the stored per-token mean and variance. Gradients with respect to the scale and bias parameters are accumulated across all tokens.

The input gradient is computed by combining contributions from normalization, variance, and mean terms, following the standard layer normalization derivative.

6.5 Embedding Gradients

Gradients with respect to token and positional embeddings are accumulated by indexing into the corresponding embedding tables. Each occurrence of a token contributes additively to its embedding gradient.

7. OPTIMIZATION & TRAINING PROCEDURE

Model parameters are updated using stochastic gradient descent with a fixed learning rate. Before each update, gradients are clipped

element-wise to a predefined threshold to prevent numerical instability.

Gradients containing non-finite values are explicitly zeroed. After each update, gradient buffers are reset to zero in preparation for the next training step.

Training is performed on a small character-level corpus using teacher forcing. The model is optimized to predict the next character in the sequence given all previous characters.

8. PERSISTENCE & EXECUTION MODEL

A central contribution of Phonex-C0 is its persistence mechanism. All model parameters are serialized into a binary file that stores tensor dimensions followed by raw floating-point values. This format is architecture-independent and can be loaded without auxiliary metadata.

At runtime, the executable operates in one of two modes: training or inference. In training mode, the model is initialized from scratch and periodically saved to disk. In inference mode, the model is reconstructed entirely from the saved binary file and used to generate text autoregressively.

This design allows the executable to function as a self-contained learning system with persistent state, enabling reproducibility and offline deployment.

9. LIMITATIONS

Phonex-C0 is intentionally constrained in scale and scope. The model uses a single attention head, a single transformer block, and a small embedding dimension. It is not optimized for performance and does not exploit hardware acceleration or parallelism.

Additionally, the implementation prioritizes clarity over numerical efficiency. As a result, training large models or long sequences is

computationally impractical within this framework.

10. CONCLUSION

Phonex-C0 demonstrates that a complete transformer language model can be implemented from first principles using only standard C. By exposing every computational and numerical detail, the system serves as both an educational artifact and a foundation for low-level research into transformer mechanics, optimization, and persistence.

Future work may extend this framework to multi-head attention, deeper architectures, alternative optimization methods, and deployment in embedded or resource-constrained environments.